"O desenvolvimento de Classes é uma atividade interessante, criativa e desafiadora, sempre com o objetivo de se criar Classes de valor" (Deitel e Deitel, 2017).

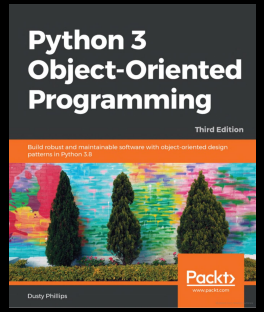
Orientação a Objetos
Parte 1

Paulo Lisboa de Almeida

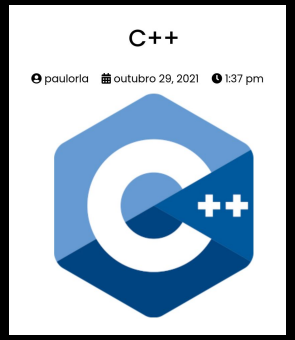
GenAI

UFPR

Leitura Recomendada



Phillips, D.. Python 3 Object-Oriented Programming.: Build Robust and Maintainable Software with Object-oriented Design Patterns in Python 3.8. 2018.



https://prlalmeida.com.br/2021/10/29/cpp

Classes e Objetos

Classes e Objetos

Uma classe possui estado (representado pelas suas variáveis) e comportamentos.

Classes e Objetos

Vamos começar definindo a estrutura do nosso projeto.

```
Diretorio_projeto (raiz)
    |
    -src
      |
      - main.py
      - universidade
```

Padrão de nome

De acordo com o PEP 8, os nomes de classes devem serguir o padrão Upper Camel Case (PEP 8 chama de CapWords).

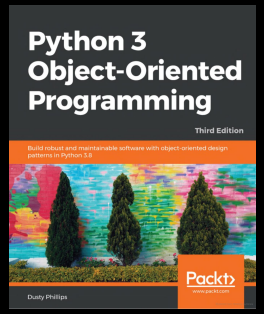
Exemplos de nome de classes:

```
Pessoa
Professor
SalaAula
```

https://peps.python.org/pep-0008/#class-names

Padrão de arquivo

Phillips, D.. Python 3 Object-Oriented Programming.: Build Robust and Maintainable Software with Object-oriented Design Patterns in Python 3.8. 2018.



Cada classe deve ser salva em seu próprio arquivo .py

O arquivo deve ter o mesmo nome da classe mas, de acordo com o PEP 8, o nome do arquivo deve seguir o padrão snake case (argh!!!).

Noma da Classe	Nome do Arquivo
Pessoa	pessoa.py
Professor	professor.py
SalaAula	sala_aula.py

Criando a Primeira Classe

Vamos criar uma classe para representar Pessoas.

No diretório src/universidade crie um arquivo pessoa.py, com o conteúdo.

```
class Pessoa:
  def __init__(self):
      self.nome = None
      self.cpf = -1
```

Criando a Primeira Classe

Vamos criar uma classe para representar Pessoas.

No diretório src/universidade crie um arquivo pessoa.py, com o conteúdo.

```
class Pessoa:
  def __init__(self):
      self.nome = None
      self.cpf = -1
```

Inicializador do Python. A maioria das linguagens de Orientadas a Objetos possuem construtores. O Python define construtor e inicializador. Comumente vamos usar o inicializador.

O inicializador é chamado automaticamente quando um objeto é criado. Veremos adiante.

Criando a Primeira Classe

Vamos criar uma classe para representar Pessoas.

No diretório src/universidade crie um arquivo pessoa.py, com o conteúdo.

```
class Pessoa:
  def __init__(self):
      self.nome = None
      self.cpf = -1
```

Self pode ser traduzido em algo como “eu mesmo”. Aqui, significa algo como “meu nome” e “meu cpf”. Ou seja, estamos nos referindo ao nome e cpf da pessoa em questão.

Criando a Primeira Classe

Vamos criar uma classe para representar Pessoas.

No diretório src/universidade crie um arquivo pessoa.py, com o conteúdo.

```
class Pessoa:
  def __init__(self):
      self.nome = None
      self.cpf = -1
```

As variáveis pertencem a Classe Pessoa.
Representam o **estado** ou os **dados** de cada objeto.

As variáveis de um objeto são comumente chamados de **atributos** ou **propriedades**.

Instanciando

Pessoa é uma classe.
Define a *estrutura* de uma pessoa.

Agora podemos criar pessoas na memória.

- Criar variáveis do tipo Pessoa.
- Essas variáveis são chamados de objeto do tipo Pessoa.
- Quando criamos um objeto, estamos **instanciando um objeto da classe**.

Instanciando

No main.py

```
from universidade.pessoa import
Pessoa


p1 = Pessoa()
p2 = Pessoa()

p1.nome = "João da Silva"
p2.nome = "Maria Oliveira"

print(p1.nome)
print(p1.cpf)

print(p2.nome)
```

Inicializador

Inicializador

O inicializador é chamado automaticamente quando um objeto é criado.

Veja você mesmo.

Inicializador

O inicializador é chamado automaticamente quando um objeto é criado.

Veja você mesmo.

Coloque um print no inicializador, ou um breakpoint e faça o Debug.

Parâmetros

Podemos incluir argumentos no inicializador.

No exemplo, tornamos obrigatório que o nome e cpf da pessoa sejam passados como parâmetro para instanciar um objeto.

```
class Pessoa:
  def __init__(self, nome, cpf):
      self.nome = nome
      self.cpf = cpf
```

Parâmetros opcionais

Da mesma forma com que qualquer função, podemos tornar parâmetros opcionais através de valores *default*.

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.nome = nome
      self.cpf = cpf
```

Getters, Setters e acesso

Público

Tanto cpf quando nome são **públicos**.

Podem ser acessados internamente na classe, ou a partir de qualquer outro arquivo .py.

pessoa.py

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.nome = nome
      self.cpf = cpf
```

main.py

```
from universidade.pessoa import Pessoa


p1 = Pessoa()

p1.nome = "João"
p1.cpf = 123

print(p1.nome)
```

Privado

Vamos usar o modificador de acesso **privado** nos atributos da classe.

Significa que o item pode ser acessado **apenas internamente na classe**.

Um membro privado em Python deve iniciar com dois underscores __.

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.__nome = nome
      self.__cpf = cpf
```

Cuidado!

Diferente de outras linguagens, no Python os modificadores de acesso são apenas uma convenção que você deve seguir.

O interpretador não faz nenhuma verificação para evitar que você acesse o que não deveria fora da Classe.

Se você o fizer, poderá gerar um programa difícil de manter, ou introduzir erros difíceis de encontrar e corrigir.

Não faça isso, apesar do interpretador deixar.

```
from universidade.pessoa import Pessoa


p1 = Pessoa()

p1.__nome = "João"
p1.__cpf = 123

print(p1.__nome)
```

Getters e Setters

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.__nome = nome
      self.__cpf = cpf

  @property
  def nome(self):
      return self.__nome

  @nome.setter
  def nome(self, nome):
      self.__nome = nome

  @property
  def cpf(self):
      return self.__cpf

  @cpf.setter
  def cpf(self, cpf):
      self.__cpf = cpf
```

Atributos (propriedades) privados dos objetos.
Ninguém fora da classe deve acessar diretamente.

Getters e Setters

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.__nome = nome
      self.__cpf = cpf

  @property
  def nome(self):
      return self.__nome

  @nome.setter
  def nome(self, nome):
      self.__nome = nome

  @property
  def cpf(self):
      return self.__cpf

  @cpf.setter
  def cpf(self, cpf):
      self.__cpf = cpf
```

`@property` é um decorador que indica que essa função serve para acessar o (atributo) nome.

A função retorna a string do nome.

Como a função acessa o atributo, chamamos de **getter**.

Getters e Setters

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.__nome = nome
      self.__cpf = cpf

  @property
  def nome(self):
      return self.__nome

  @nome.setter
  def nome(self, nome):
      self.__nome = nome

  @property
  def cpf(self):
      return self.__cpf

  @cpf.setter
  def cpf(self, cpf):
      self.__cpf = cpf
```

Similar ao getter, mas aqui indicamos que esse é um **setter**.

Recebe um nome como parâmetro, e o atribui para o nome (privado) da pessoa.

Getters e Setters

pessoa.py

```python
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.__nome = nome
      self.__cpf = cpf

  @property
  def nome(self):
      return self.__nome

  @nome.setter
  def nome(self, nome):
      self.__nome = nome

  @property
  def cpf(self):
      return self.__cpf

  @cpf.setter
  def cpf(self, cpf):
      self.__cpf = cpf
```

main.py

```python
from universidade.pessoa import Pessoa


p1 = Pessoa()

p1.nome = "João"
p1.cpf = 123

print(p1.nome)
```

Faça você mesmo

Adicione prints ou breakpoints nos getters e setters e verifique que eles realmente são chamados!

Encapsulamento

CPFs válidos

Os dois últimos dígitos de um CPF são validadores.

Indicam se um CPF é válido ou não.

Existe um algoritmo que faz essa verificação. Pesquise na internet.

Algoritmo em Python

```python
def __validar_cpf(self, cpf_teste):
    somatorio_valida_ultimo = 0
    somatorio_valida_penultimo = 0

    ultimo = cpf_teste % 10
    cpf_teste //= 10
    penultimo = cpf_teste % 10
    cpf_teste //= 10

    somatorio_valida_ultimo = penultimo * 2
    for i in range(2, 12):
        modulo = cpf_teste % 10
        cpf_teste //= 10
        somatorio_valida_penultimo += modulo * i
        somatorio_valida_ultimo += modulo * (i + 1)

    modulo = somatorio_valida_penultimo % 11
    if modulo < 2:
        if penultimo != 0:
            return False  # CPF inválido
    else:
        if penultimo != 11 - modulo:
            return False  # CPF inválido

    modulo = somatorio_valida_ultimo % 11
    if modulo < 2:
        if ultimo != 0:
            return False  # CPF inválido
    else:
        if ultimo != 11 - modulo:
            return False  # CPF inválido

    return True  # CPF válido
```

Validando

Um objeto da classe Pessoa deve ser capaz de validar o CPF sendo setado.

Apesar de incorreto no português, comumente utilizamos esse anglicismo quando estamos alterando o atributo através do *setter*.

Validando

Um objeto da classe Pessoa deve ser capaz de validar o CPF sendo setado.

O validador de CPF está privado.

Princípio do encapsulamento.

Quem usar nossa classe não precisa, nem deve, conhecer os detalhes internos da classe.

Pessoa

Inicializador e setter verificam automaticamente se o CPF é válido.

Se não for, insere -1 para indicar que o usuário tentou carregar um CPF inválido.

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.__nome = nome
      if cpf != -1 and self.__validar_cpf(cpf):
          self.__cpf = cpf
      else:
          self.__cpf = -1 #CPF inválido

  @property
  def cpf(self):
      return self.__cpf

  @cpf.setter
  def cpf(self, cpf):
      if self.__validar_cpf(cpf):
          self.__cpf = cpf
      else:
          self.__cpf = -1 #Indica CPF inválido
```

Faça você mesmo

Tente fazer você mesmo a classe Pessoa do zero, sem olhar os slides.

Pesquise na internet sobre como um CPF é validado, e implemente sua versão do algoritmo.

Associações

Disciplina

Vamos criar uma classe que representa uma disciplina.

```
class Disciplina:
  def __init__(self, nome, carga_horaria = 60):
      self.__nome = nome
      if carga_horaria > 0:
          self.__carga_horaria = carga_horaria
      else:
          self.__carga_horaria = -1 #Carga inválida
   @property
  def nome(self):
      return self.__nome

  @nome.setter
  def nome(self, nome):
      self.__nome = nome

  @property
  def carga_horaria(self):
      return self.__carga_horaria

  @carga_horaria.setter
  def carga_horaria(self, carga_horaria):
      if carga_horaria > 0:
          self.__carga_horaria = carga_horaria
      else:
          self.__carga_horaria = -1 #Carga Inválida
```

Professor

Uma disciplina deve ter um professor responsável por ministrá-la.

Isso será representado por um objeto do tipo Pessoa dentro de Disciplina.

Professor

```
class Disciplina:
  def __init__(self, nome, carga_horaria = 60, professor=None):
      self.__nome = nome
      if carga_horaria > 0:
          self.__carga_horaria = carga_horaria
      else:
          self.__carga_horaria = -1 #Carga inválida
      self.__professor = professor

  @property
  def professor(self):
      return self.__professor

  @professor.setter
  def professor(self, professor):
      self.__professor = professor
```

main.py

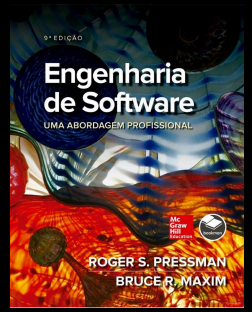
```python
from universidade.pessoa import Pessoa
from universidade.disciplina import Disciplina


p1 = Pessoa("João",11111111111)
d1 = Disciplina("Orientação a Objetos", 60, p1)


print(d1.nome)
print(d1.carga_horaria)
print(d1.professor.nome)
```

Associação

Pressman, R., Bruce, R. M.Engenharia de Software - Uma Abordagem Profissional. 9a ed. 2021.



A relação entre as classes Disciplina e Pessoa é uma associação.

Algumas propriedades de uma associação:

- O objeto associado (Pessoa) não tem relação direta com a classe (Disciplina).
- O objeto associado (Pessoa) pode pertencer a mais de uma relação ao mesmo tempo.
- A classe (Disciplina) não gerencia a existência do objeto associado.
- A pessoa que representa o professor da disciplina existe independentemente da classe e objetos Disciplina.
- O dado membro associado (Pessoa) pode ou não saber da existência da classe (Disciplina).

Comportamentos

Exibir Informações

```
class Disciplina:

  #...

  @carga_horaria.setter
  def carga_horaria(self, carga_horaria):
      if carga_horaria > 0:
          self.__carga_horaria = carga_horaria
      else:
          self.__carga_horaria = -1 #Carga Inválida

  def exibir_informacoes(self):
      print("Disciplina:", self.__nome)
      print("Carga Horária:", self.__carga_horaria)
      if self.__professor is not None:
          print("Professor Responsável:", self.__professor.nome)
      else:
          print("Disciplina sem professor alocado")
```

Exibir Informações

Podemos adicionar quantas funções úteis à classe desejarmos.

As funções atribuídas a classe são comumente chamadas de **métodos**.

```
class Disciplina:

  #...

  @carga_horaria.setter
  def carga_horaria(self, carga_horaria):
      if carga_horaria > 0:
          self.__carga_horaria = carga_horaria
      else:
          self.__carga_horaria = -1 #Carga Inválida

  def exibir_informacoes(self):
      print("Disciplina:", self.__nome)
      print("Carga Horária:", self.__carga_horaria)
      if self.__professor is not None:
          print("Professor Responsável:", self.__professor.nome)
      else:
          print("Disciplina sem professor alocado")
```

Exibir Informações

Podemos adicionar quantas funções úteis à classe desejarmos.

As funções atribuídas a classe são comumente chamadas de **métodos**.

Os métodos definem o **comportamento** dos objetos.

```
class Disciplina:

  #...

  @carga_horaria.setter
  def carga_horaria(self, carga_horaria):
      if carga_horaria > 0:
          self.__carga_horaria = carga_horaria
      else:
          self.__carga_horaria = -1 #Carga Inválida

  def exibir_informacoes(self):
      print("Disciplina:", self.__nome)
      print("Carga Horária:", self.__carga_horaria)
      if self.__professor is not None:
          print("Professor Responsável:", self.__professor.nome)
      else:
          print("Disciplina sem professor alocado")
```

Dados Estáticos

Conteúdo Ministrado

```python
class ConteudoMinistrado:
  def __init__(self, descricao, carga_horaria):
      self.__descricao = descricao
      self.__carga_horaria = carga_horaria

  @property
  def descricao(self):
      return self.__descricao

  @property
  def carga_horaria(self):
      return self.__carga_horaria
```

Disciplina

```python
from .conteudo_ministrado import ConteudoMinistrado

class Disciplina:
  def __init__(self, nome, carga_horaria = 60, professor=None):
      self.__nome = nome
      if carga_horaria > 0:
          self.__carga_horaria = carga_horaria
      else:
          self.__carga_horaria = -1 #Carga inválida
      self.__professor = professor
      self.__conteudos_ministrados = []

  @property
  def conteudos_ministrados(self):
      return self.__conteudos_ministrados

  def adicionar_conteudo_ministrado(self, conteudo, carga):
      self.__conteudos_ministrados.append(ConteudoMinistrado(conteudo, carga))

  def exibir_informacoes(self):
      print("Disciplina:", self.__nome)
      print("Carga Horária:", self.__carga_horaria)
      if self.__professor is not None:
          print("Professor Responsável:", self.__professor.nome)
      else:
          print("Disciplina sem professor alocado")

      print("Conteúdos Ministrados:")
      for cont in self.__conteudos_ministrados:
          print(f"\t->{cont.descricao}({cont.carga_horaria} hora)")
```

main.py

```python
from universidade.pessoa import Pessoa
from universidade.disciplina import Disciplina
from universidade.conteudo_ministrado import ConteudoMinistrado


p1 = Pessoa("João",11111111111)
d1 = Disciplina("Orientação a Objetos", 60, p1)
d1.adicionar_conteudo_ministrado("Getters e Setters", 1)
d1.adicionar_conteudo_ministrado("Classes", 2)
d1.adicionar_conteudo_ministrado("Modificadores de Acesso", 1)


d1.exibir_informacoes()
```

Identificador Único

Desejamos que cada conteúdo ministrado tenha um identificador em formato de um número.

Vai ser o “CPF” do conteúdo, que vai ser único para cada conteúdo ministrado.

Desejamos que a classe *ConteudoMinistrado* gerencie esses identificadores automaticamente.

Identificador Único

```
class ConteudoMinistrado:
  def __init__(self, descricao, carga_horaria):
      self.__descricao = descricao
      self.__carga_horaria = carga_horaria
      self.__id = 0

  @property
  def descricao(self):
      return self.__descricao

  @property
  def carga_horaria(self):
      return self.__carga_horaria

  @property
  def id(self):
      return self.__id
```

Dado estático

Os dados que criamos até agora pertencem aos objetos.

Cada objeto do tipo ConteudoMinistrado criado na memória possui sua própria descrição, carga e id.

```
class ConteudoMinistrado:
  def __init__(self, descricao, carga_horaria):
      self.__descricao = descricao
      self.__carga_horaria = carga_horaria
      self.__id = 0
```

Dado estático

Os dados que criamos até agora pertencem aos objetos.

Cada objeto do tipo ConteudoMinistrado criado na memória possui sua própria descrição, carga e id.

Uma **dado estático** não pertence aos objetos, mas sim a **classe**.

“Compartilhado” entre todos os objetos do mesmo tipo.

Dado estático

Coloque as variáveis estáticas antes dos métodos.

Uma variável estática **não começa com self**.

```
class ConteudoMinistrado:
    PROX_ID = 0
    def __init__(self, descricao, carga_horaria):
        self.__descricao = descricao
        self.__carga_horaria = carga_horaria
        self.__id = 0

    @property
    def descricao(self):
        return self.__descricao

    @property
    def carga_horaria(self):
        return self.__carga_horaria

    @property
    def id(self):
        return self.__id
```

Dado estático

Na falta de uma recomendação melhor, vamos usar o estilo PEP8 para constantes (não é a mesma coisa) para dados estáticos.
Tudo maiúsculo separado por underscores.

```
class ConteudoMinistrado:
    PROX_ID = 0
    def __init__(self, descricao, carga_horaria):
        self.__descricao = descricao
        self.__carga_horaria = carga_horaria
        self.__id = 0

    @property
    def descricao(self):
        return self.__descricao

    @property
    def carga_horaria(self):
        return self.__carga_horaria

    @property
    def id(self):
        return self.__id
```

Acessando

Para acessar a variável estática, use o nome da classe seguido de ponto.

main.py

```
from universidade.pessoa import Pessoa
from universidade.disciplina import Disciplina
from universidade.conteudo_ministrado import ConteudoMinistrado


conteudo1 = ConteudoMinistrado("Conteudo1", 1)
ConteudoMinistrado.prox_id = 56
conteudo2 = ConteudoMinistrado("Conteudo2", 2)
ConteudoMinistrado.prox_id = 999

print(conteudo1.descricao)
print(conteudo2.descricao)

print(conteudo1.prox_id)
print(conteudo2.prox_id)
```

Funciona, mas é uma **má prática**.
Dá a entender que estamos acessando um atributo do objeto.

Acessando

Para acessar a variável estática, use o nome da classe seguido de ponto.

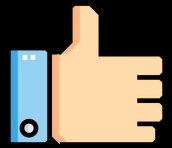
main.py

```
from universidade.pessoa import Pessoa
from universidade.disciplina import Disciplina
from universidade.conteudo_ministrado import ConteudoMinistrado


conteudo1 = ConteudoMinistrado("Conteudo1", 1)
ConteudoMinistrado.prox_id = 56
conteudo2 = ConteudoMinistrado("Conteudo2", 2)
ConteudoMinistrado.prox_id = 999

print(conteudo1.descricao)
print(conteudo2.descricao)

print(ConteudoMinistrado.prox_id)
print(ConteudoMinistrado.prox_id)
```



Gerando ids

```
class ConteudoMinistrado:
  __PROX_ID = 0

  def __init__(self, descricao, carga_horaria):
      self.__descricao = descricao
      self.__carga_horaria = carga_horaria
      self.__id = ConteudoMinistrado.__PROX_ID
      ConteudoMinistrado.__PROX_ID += 1

   @property
   def descricao(self):
       return self.__descricao

   @property
   def carga_horaria(self):
       return self.__carga_horaria

   @property
   def id(self):
       return self.__id
```

Modificando a exibição

```python
from .conteudo_ministrado import ConteudoMinistrado

class Disciplina:

    #...

    def exibir_informacoes(self):
        print("Disciplina:", self.__nome)
        print("Carga Horária:", self.__carga_horaria)
        if self.__professor is not None:
            print("Professor Responsável:", self.__professor.nome)
        else:
            print("Disciplina sem professor alocado")

        print("Conteúdos Ministrados:")
        for cont in self.__conteudos_ministrados:
            print(f"\t->id: {cont.id} {cont.descricao} ({cont.carga_horaria} hora)")
```

Testando no main.py

Console

```
Disciplina: Orientação a Objetos
Carga Horária: 60
Professor Responsável: João
Conteúdos Ministrados:
        ->id: 0 Getters e Setters (1 hora)
        ->id: 1 Classes (2 hora)
        ->id: 2 Modificadores de Acesso (1 hora)
```

main.py

```python
from universidade.pessoa import Pessoa
from universidade.disciplina import Disciplina
from universidade.conteudo_ministrado import ConteudoMinistrado


p1 = Pessoa("João",11111111111)
d1 = Disciplina("Orientação a Objetos", 60, p1)
d1.adicionar_conteudo_ministrado("Getters e Setters", 1)
d1.adicionar_conteudo_ministrado("Classes", 2)
d1.adicionar_conteudo_ministrado("Modificadores de Acesso", 1)


d1.exibir_informacoes()
```

Cuidado!

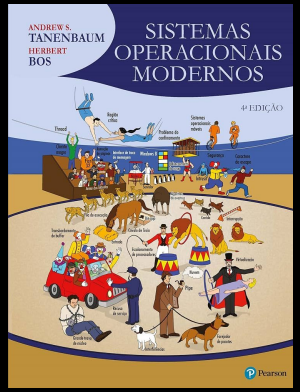
O exemplo funciona assumindo que **não temos condições de corrida**.

Se seu programa executa localmente, por exemplo, e em uma única thread, não teremos problemas.

Programas com múltiplas threads ou processos precisam tratar o incremento da variável estática.

Exemplo: através de mutexes ou semáforos.

Veja em livros de Sistemas Operacionais.



Tanenbaum, A. S., Bos, H. Modern Operating Systems. 2015.

Faça você mesmo

Aprendemos sobre atributos estáticos.

Estude sobre o conceito de métodos estáticos em Python.

Fim!

É isso!

Licença

Esta obra está licenciada com uma Licença Creative Commons Atribuição 4.0 Internacional.